

Late Delivery Prediction in Supply Chain Logistics Using Machine Learning Models

Sachin Kirtibhai Patel
Patel.1801@wright.edu

Abstract—Late delivery is a common problem in supply chain operations because it can affect service quality and customer satisfaction. This project uses machine learning models to predict whether an order will be delivered late or on time using the DataCo Smart Supply Chain dataset. After data cleaning, removal of leakage columns, and feature engineering, a stratified sample of 50,000 records was used for model training and testing. Three classification models were compared: Logistic Regression, Decision Tree, and Random Forest. Performance was evaluated using accuracy, precision, recall, F1 score, and ROC-AUC. Results showed that tree-based models performed better than Logistic Regression, with Random Forest giving the strongest overall performance. The analysis also indicated that shipping mode and the engineered ship_gap feature were important predictors of late delivery risk. These results show machine learning can support early identification of risky shipments.

Keywords—late delivery prediction, supply chain, machine learning, classification, random forest, decision tree, logistic regression

GitHub Link: <https://github.com/Sachin1610/smart-logistics-delay-prediction>

I. INTRODUCTION

Late delivery is a common problem in supply chain and e-commerce because delays can affect customer satisfaction and business operations. Predicting which orders may be at risk of being delivered late can help support better logistics decisions.

Traditional shipment monitoring often identifies problems only after delays occur. Machine learning offers a way to predict late deliveries earlier by using historical order data [5], [6]. Features such as shipping mode, order information, customer location, and processing time can help identify patterns related to delivery delays.

In this project, the DataCo Smart Supply Chain dataset [1] is used to predict whether an order will be delivered late. Three machine learning models were compared: Logistic Regression, Decision Tree, and Random Forest [2], [7]. Hyperparameters were tuned using grid search with cross-validation.

The goal of this work is to evaluate whether classical machine learning models, combined with preprocessing and feature engineering, can effectively predict late delivery risk in supply chain operations.

II. RELATED WORK

Machine learning has been used in earlier studies for problems in supply chain and logistics, including forecasting, shipment risk prediction, and delivery delay prediction [5], [6]. Models such as Logistic Regression, Decision Tree, and Random Forest have been commonly used because they perform well on structured data [2], [7].

Some studies have shown that tree-based models can capture relationships in logistics data better than simpler linear models [2], [3]. Features such as shipping mode, processing time, and order information have also been reported as useful factors related to delivery delays. This project follows a similar idea by comparing three supervised learning models on the DataCo Smart Supply Chain dataset, with focus on preprocessing, feature engineering, and model performance evaluation [1], [5].

Some prior studies have also reported that feature engineering and data preparation can have a major effect on model performance in logistics problems [4], [7]. This aligns with the findings in this project, where engineered features contributed strongly to prediction results.

III. METHODOLOGY

A. Problem Formulation

This work is treated as a binary classification problem. Given features such as shipping mode, product category, payment type, customer location, order timing, and warehouse processing time, the goal is to predict whether an order will be delivered late or on time. The target variable is Late_delivery_risk, where 1 represents a late delivery and 0 represents an on-time delivery. The dataset has about 54.8% late orders and 45.2% on-time orders. Models are trained using historical data and evaluated on a separate test set.

Since each order must be classified as either late or on time, this problem is naturally modeled as a binary classification task. Each model estimates the relationship between the input features and the likelihood of delay, then assigns a class label based on prediction output. This setup allows comparison across different classification methods using the same data and evaluation criteria.

B. Models Used

Three machine learning models were used in this project: Logistic Regression, Decision Tree, and Random Forest. These models were selected because they are commonly used for classification tasks, work well on structured tabular data, and provide a useful comparison between linear, non-linear, and ensemble methods. LogisticRegression was used as a baseline model to estimate the probability of late delivery from the input features. Decision Tree was used to capture non-linear patterns in the data through feature-based splits. Random Forest was used to improve prediction performance and reduce overfitting compared to a single tree.

Random Forest also provides feature importance scores that help identify useful predictors. All models were implemented using scikit-learn in Python. The models also provide a useful comparison in behavior. Logistic Regression offers a simple baseline, Decision Tree captures non-linear patterns, and Random Forest improves stability by combining multiple trees. This comparison helps show whether added model complexity improves prediction performance.

C. Training Procedure

For each model, hyperparameter tuning was performed using GridSearchCV with 3-fold cross-validation. F1 score was used as the model selection metric because it balances precision and recall. This process was used to select the best parameter settings for each model. For Logistic Regression, the regularization parameter C was tuned. For Decision Tree and Random Forest, parameters such as tree depth, split size, and leaf size were tuned. L2 regularization was used for Logistic Regression, and multiple parameter combinations were evaluated during model tuning.

After selecting the best parameters, each model was trained using the training set and evaluated on the validation set during model selection. Final evaluation was then performed on the held-out test set using accuracy, precision, recall, F1 score, and ROC-AUC. All experiments were implemented in Python using pandas, NumPy, and scikit-learn. Random seed 42 was used throughout the experiments to support reproducibility. Using cross-validation during tuning helped reduce overfitting and improved confidence in model selection. This approach ensured that the final test results were based on unseen data and reflect model performance on new unseen orders.

IV. EXPERIMENTAL SETUP

A. Datasets

Experiments were done using the DataCo Smart Supply Chain dataset, which contains 180,519 order records and 53 features related to orders, shipping, customers, and products. The target variable is `Late_delivery_risk`, where 1 means a late delivery and 0 means an on-time delivery. Before training the models, preprocessing was done to remove leakage columns, redundant features, and columns with many missing values. Date-based features were also created, including `Order_Hour`, `Order_DayOfWeek`, `Order_Month`, `Order_Year`, `Is_Weekend`, and the engineered `ship_gap` feature. For modeling, a stratified sample of 50,000 records was used while keeping the same class distribution. The data was split into training, validation, and test sets using a 70/15/15 ratio, and categorical variables were converted using one-hot encoding.

TABLE I. Dataset Summary

Metric	Value
Records Used	50,000
Original Features	53
Features After Cleaning	25
Features After Encoding	110
Late Deliveries	54.8%
On-Time Deliveries	45.2%
Train/Val/Test Split	70/15/15

B. Evaluation Metrics

Model performance was evaluated using Accuracy, Precision, Recall, F1 score, and ROC-AUC. Accuracy measures overall correct predictions, but because the classes are slightly imbalanced, other metrics were also used. Precision shows how reliable a late prediction is, meaning out of all orders predicted as late, how many were actually late. Recall shows how many actual late deliveries were correctly identified by the model, which is important because missing a risky order can be costly. F1 score balances Precision and Recall, making it useful when both false positives and false negatives matter. ROC-AUC was used to measure how well the model separates the two classes across different decision thresholds.

C. Implementation Details

All experiments were done in Python using pandas, NumPy, scikit-learn, matplotlib, and seaborn. Pandas and NumPy were used for data handling, scikit-learn was used for preprocessing, model training, GridSearchCV, and evaluation, and matplotlib and seaborn were used for plots. Logistic Regression used scaled numerical features with StandardScaler, while Decision Tree and Random Forest used the original unscaled data. Hyperparameter tuning was done using GridSearchCV with 3-fold cross-validation. A random seed of 42 was used so results remain consistent across runs.

V. RESULTS

A. Model Performance

Table II shows the performance of all three models on the test set. All models performed much better than the baseline accuracy of 54.83%, which would come from always predicting the majority class. Random Forest achieved the best overall performance, followed closely by Decision Tree, while Logistic Regression had slightly lower results.

Random Forest gave the strongest F1 score and ROC-AUC, while Decision Tree performed very close to it. Logistic Regression was slightly lower, suggesting tree-based models captured the data patterns better. All three models also showed high recall, which is important because missing a late delivery can be costly.

TABLE II. Test Set Performance Comparison

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.9632	0.9532	0.9810	0.9669	0.9795
Decision Tree	0.9771	0.9603	0.9995	0.9795	0.9934
Random Forest	0.9776	0.9607	1.0000	0.9800	0.9945

B. Feature Importance Analysis

Feature importance scores from the Random Forest model were used to identify which variables contributed most to late delivery prediction. Figure 2 shows the top 15 important features. The top three features were `ship_gap`, Days for shipment (scheduled), and Shipping Mode. Among them, `ship_gap` was the most important predictor.

An additional observation is that the most important features were mainly operational variables rather than customer or product variables. This suggests late delivery risk in this dataset was driven more by fulfillment conditions than by who placed the order or what product was ordered. This finding may be useful when selecting features for similar logistics prediction tasks.



Fig. 1. Late delivery rates across shipping modes.

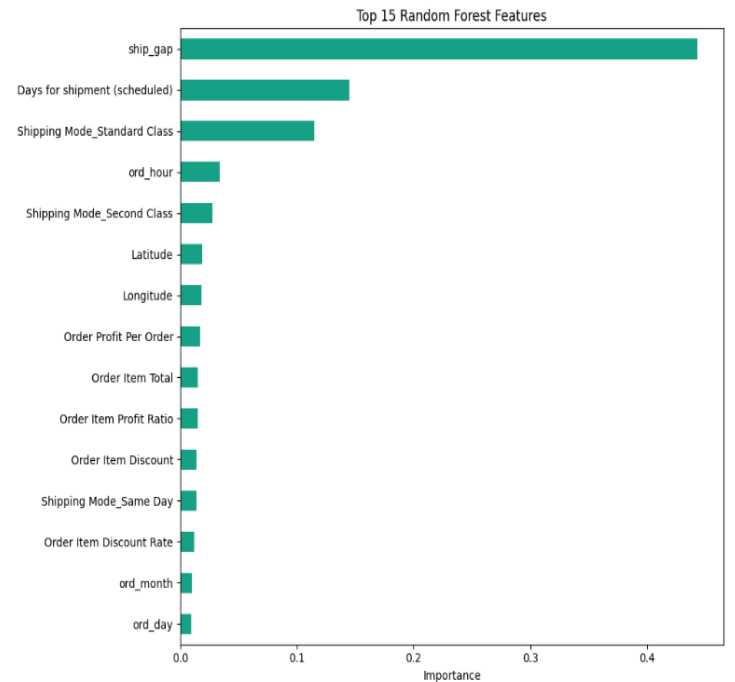


Fig. 2. Top 15 feature importance scores from Random Forest.

These results suggest that operational features related to shipping speed and processing time had more influence on prediction than customer or financial features. Variables such as product price, order quantity, and location-related features showed relatively low importance. This indicates that late delivery risk was driven more by shipping conditions than by product or customer characteristics.

C. Confusion Matrix Analysis

Figure 3 shows the confusion matrices for the three models on the test set. All three models predicted most orders correctly, while Random Forest made the fewest mistakes. Decision Tree was very close to Random Forest, and Logistic Regression had slightly more errors.

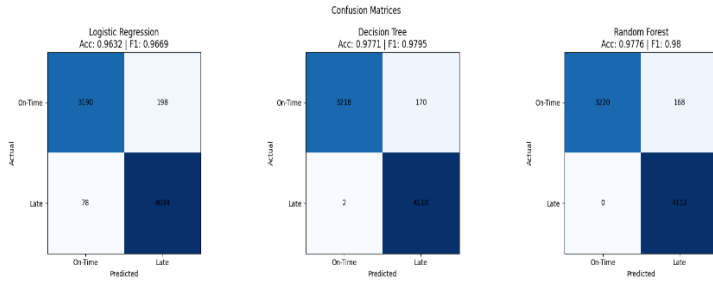


Fig. 3. Confusion matrices for the three models.

These results are consistent with the performance metrics in Table II. Overall, the confusion matrices show that Random Forest performed best, while both tree-based models performed a little better than Logistic Regression. Figure 5 shows the ROC curves for all three models. Random Forest and Decision Tree have the highest AUC values, while Logistic Regression is slightly lower.

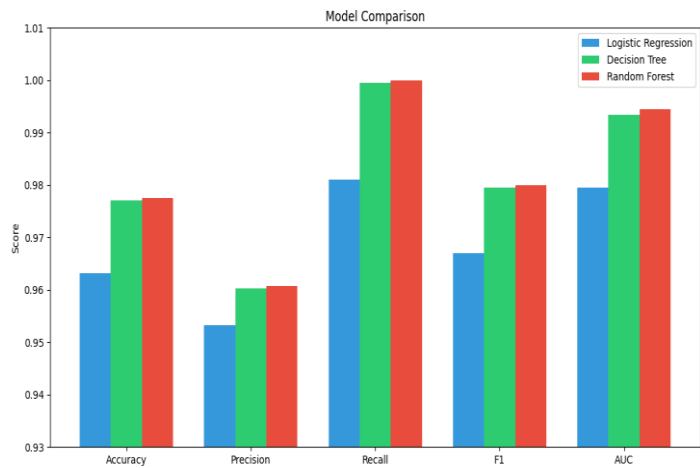


Fig. 4. Comparison of model performance metrics.

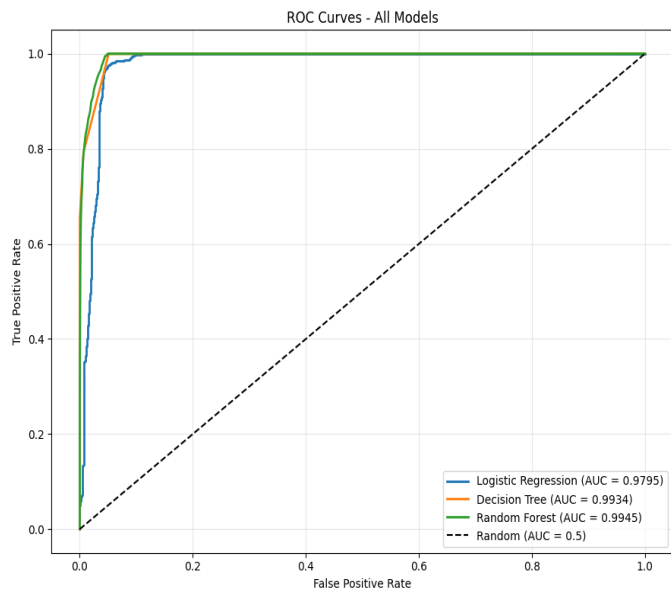


Fig. 5 ROC Curves

D. Error Analysis

Although model performance was strong, some misclassifications still occurred. Most errors came from on-time orders that were predicted as late, rather than missed late deliveries. This suggests the models were slightly conservative and tended to favor identifying risk. For this problem, that trade-off may be acceptable, but it also shows opportunities to improve precision.

Some of these errors may come from cases where orders have similar feature patterns but different outcomes. Additional features related to warehouse operations or carrier performance may help reduce such errors.

VI. DISCUSSION

The results show that all three models were effective for predicting late deliveries, with Random Forest giving the best overall performance and Decision Tree performing very close behind. This suggests the main predictive patterns in the data were captured well by tree-based models. Logistic Regression performed slightly lower, likely because the relationships in the data were not fully linear.

Feature importance analysis showed that `ship_gap` had a strong influence on prediction performance. This suggests that feature engineering played an important role in the results. One limitation is that the models were tested on a single dataset, so performance may differ on other logistics data. In addition, strong correlation between `ship_gap` and the target may have made the prediction task easier than some real-world settings.

From a practical perspective, these results suggest the models could be used to identify risky shipments before delivery deadlines are missed. For example, orders predicted as high-risk could be reviewed earlier, assigned different shipping options, or monitored more closely. This could help logistics teams reduce delays and improve service decisions. Although this project focuses on prediction, it also shows how machine learning could support operational planning in supply chain environments.

VII. CONCLUSION

This project studied late delivery prediction in supply chain logistics using the DataCo Smart Supply Chain dataset. After preprocessing and feature engineering, three machine learning models were trained and compared. Random Forest achieved the best overall performance, while Decision Tree also performed very well.

The results show that classical machine learning models can be effective for predicting late delivery risk when supported by useful

features. The ship_gap feature was found to be an important predictor, showing the value of feature engineering. Future work could test the models on other datasets and compare additional methods..

REFERENCES

- [1] S. Komati, "DataCo Supply Chain Dataset," Kaggle, 2023. [Online]. Available: <https://www.kaggle.com/datasets/saicharankomati/dataco-supply-chain-dataset>
- [2] L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.
- [3] C. M. Bishop, Pattern Recognition and Machine Learning. New York, NY, USA: Springer, 2006.
- [4] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [5] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [6] G. Wang, A. Gunasekaran, E. W. T. Ngai, and T. Papadopoulos, "Big Data Analytics in Logistics and Supply Chain Management," International Journal of Production Economics, vol. 176, pp. 98–110, 2016.
- [7] G. James, D. Witten, T. Hastie, and R. Tibshirani, An Introduction to Statistical Learning. New York, NY, USA: Springer, 2013.

AUTHOR CONTRIBUTIONS

Sachin Kirtibhai Patel is the sole author of this paper and was responsible for dataset selection, preprocessing, feature engineering, model training and tuning, results analysis, visualization, and manuscript writing. The author also developed the Python implementation and generated all tables and figures included in this work.